Mining Software Repositories

Chapter 6: Studying bugs

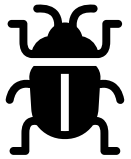
Prof. Dr. Steffen Herbold

Contents

- Bugs and MSR
- Defect prediction in general
- Costs of defect prediction
- Relationship between internal and external quality

Bugs and MSR

Understanding bugs



- No developer writes code without defects
- No non-trivial piece of software is without defects
  - Possible exception: systems in which millions or more are invested into proving the quality as is, e.g., often the case for space exploration
- Data about defects is often tracked
  - In changes through commits and their (descriptive) messages
  - In issue tracking systems
  - In pull requests
  - …
- MSR is a very good tool to investigate the nature of defects!

Martin Fowler, Kent Beck: Refactoring: Improving the Design of Existing Code, Pearson Addison-Wesley

MSR and defects

- We already covered data-related aspects regarding defects
  - Identifying bug fixing commits
  - SZZ algorithm to identify bug introducing changes

- We did not yet cover what we can do with such data
  - Localize defects, e.g., based on an issue description
    - Called bug localization in the literature
  - Learn to identify defects within code
    - Called defect prediction in the literature
  - Learn to fix defects based on the bug fixing commits
    - Called program repair in the literature and often seen as a topic independent of MSR, as there are also non-MSR based approaches for this

- Within this lecture, we focus on defect prediction

Defect prediction in general

Overview of defect prediction

Input:
Software



Output:
Prediction of defects



Reaction:
Correction of defects



Result:
Less post-release defects









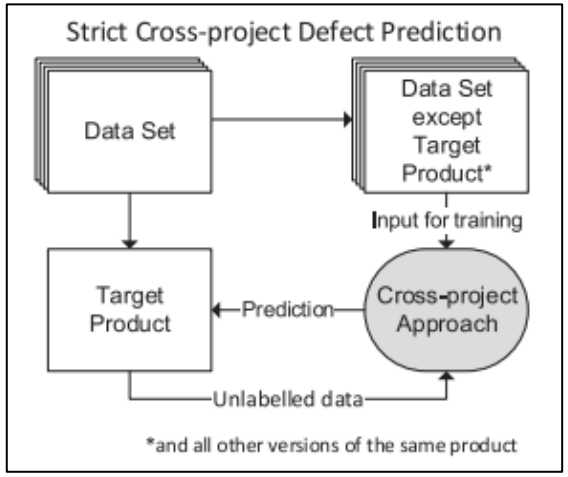
Basic terminology

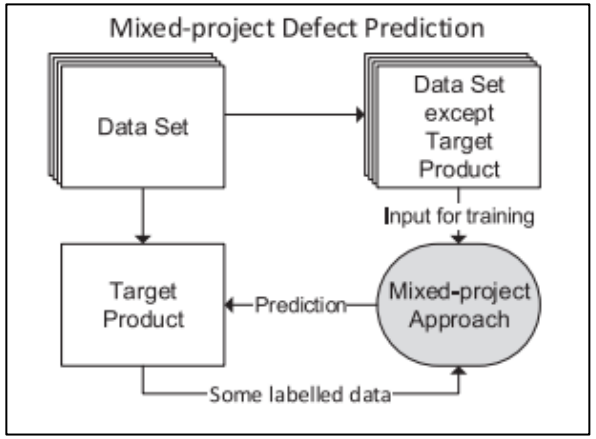
- Target project
  - The project for which we want to predict defects
- Training project(s)
  - The project(s) from which we obtain data to train a defect prediction model
- Defect prediction model
  - A machine learning model trained on historic data about defects
- Performance metric
  - A metric that can be used to assess how good a defect prediction model is at finding defects

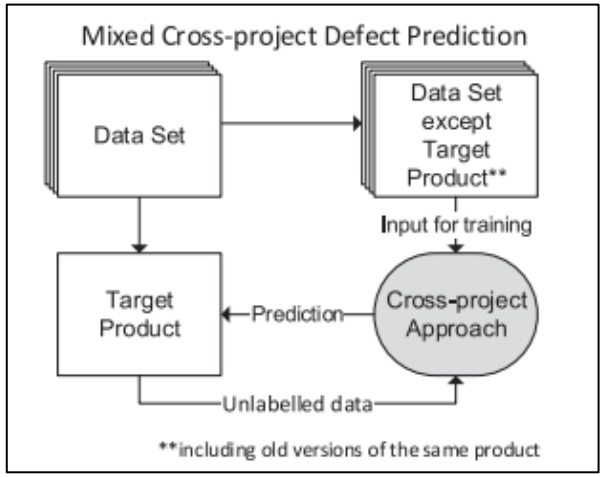
General strategies for defect prediction

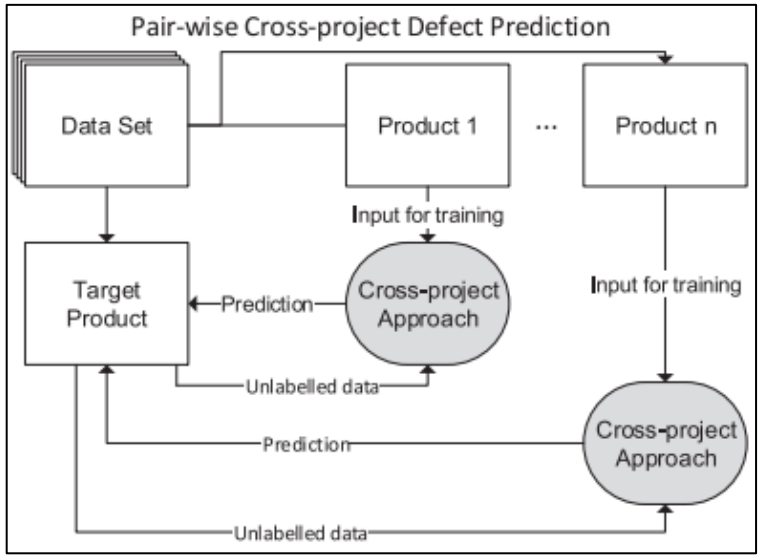
- Defect prediction is diverse
  - For which artifacts to we predict if they are defective?
    - Files?
    - Classes?
    - Methods?
    - Lines?
    - …?
  - Where do we get our training data?
    - Cross-project: training data only from other projects
    - Within-project: training data from the same project
    - …
  - When do we predict defects?
    - When we prepare a release (or release candidate) to organize QA activities?
      - Called release-level defect prediction in the literature
    - For every commit?
      - Called just-in-time defect prediction in the literature
    - …

Example: cross-project defect prediction







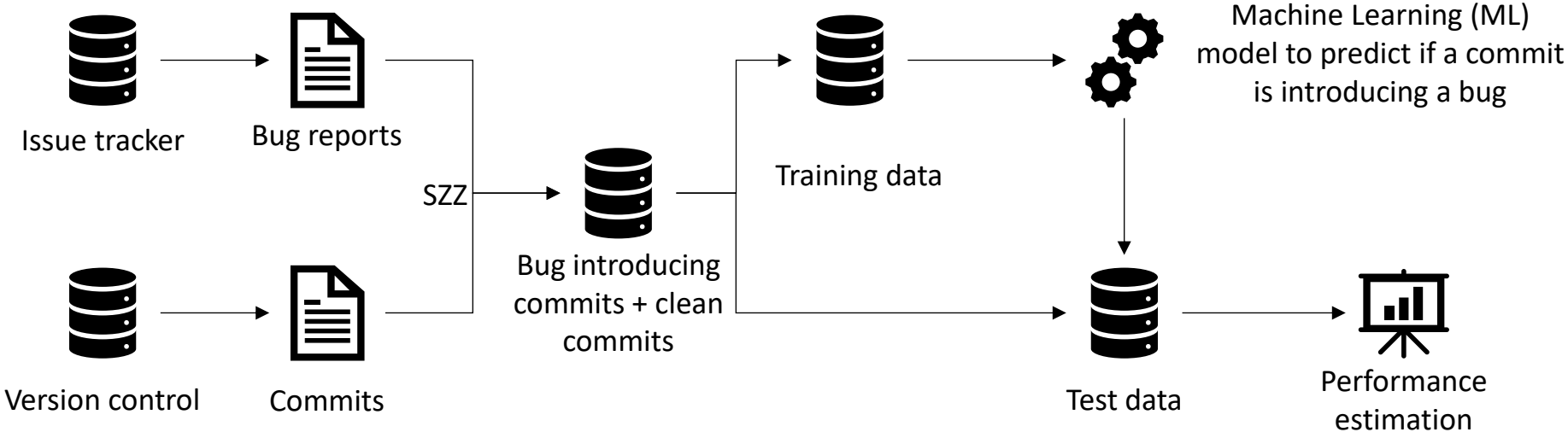


- Product = Release of a Project
- Each variant with different implications for possible usage scenarios
- Some scenarios unrealistic
  - On this slide, the pair-wise scenario would lead to multiple predictions models per target product
  - In reality, we can only one model or need a strategy to combine multiple models into a single output

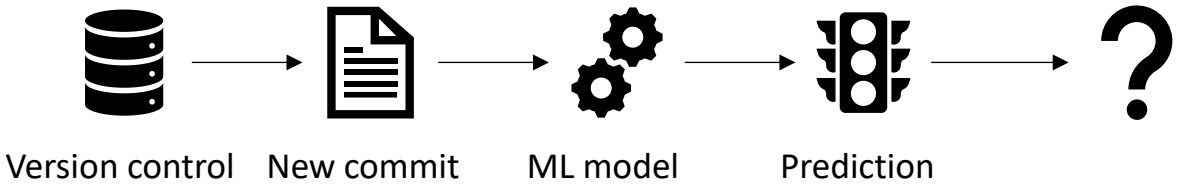
S. Herbold, A. Trautsch and J. Grabowski, "A Comparative Study to Benchmark Cross-Project Defect Prediction Approaches," in IEEE Transactions on Software Engineering, vol. 44, no. 9, pp. 811-833, 1 Sept. 2018, doi: 10.1109/TSE.2017.2724538.

Example: Just-in-time (JIT) defect prediction

Model training:



Model deployment:



Some action within the development process:

- E.g., extensive code review or testing in case of prediction as bug
- E.g., only rudimentary quality checks in case of prediction as clean

Model monitoring

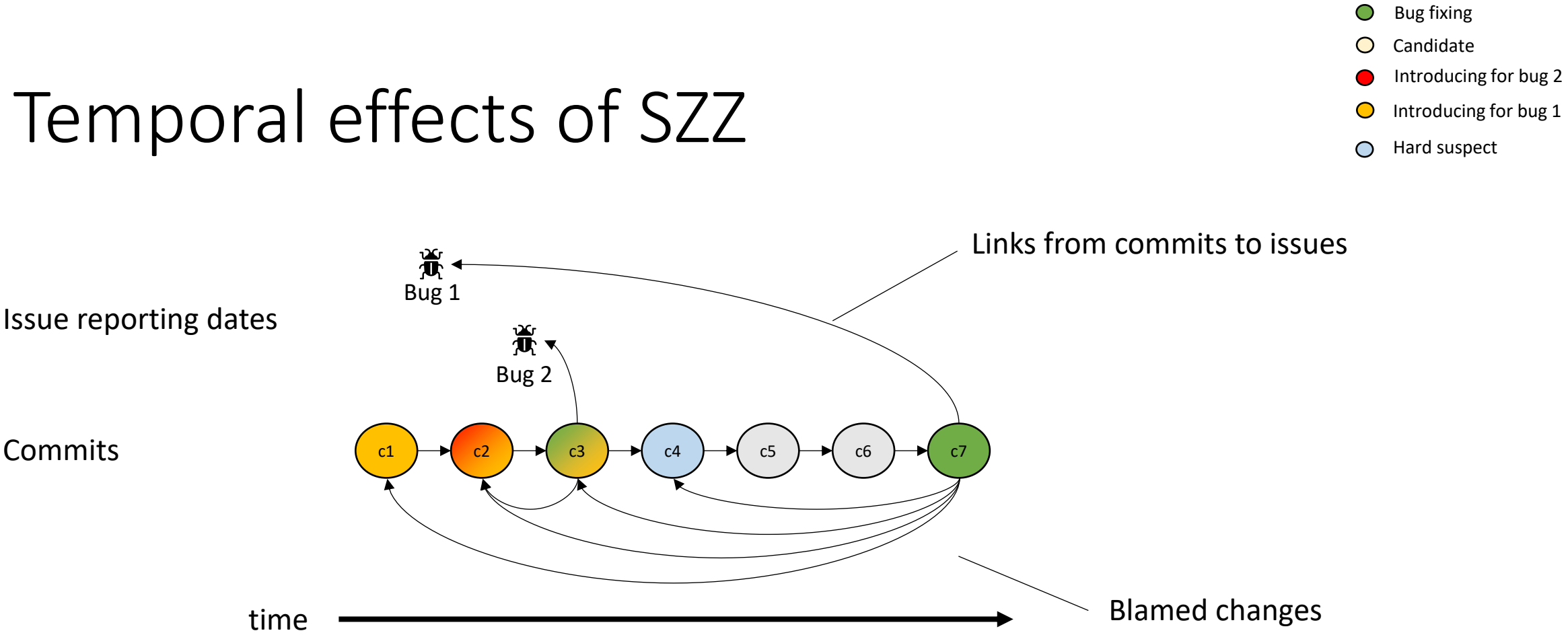
Temporal aspects for JIT defect prediction

- When working with JIT, many aspects need to be considered
- Most importantly, this is a process that should work in a real-time manner
- This needs to be considered when we create our model
  - Mostly affects data creation
- How do we create training and test data
  - Data has a temporal dimension, i.e., commits arrive in a certain order
  - We need to consider this when creating training and test data
    - Test data should never be older than training data
    - If this were the case, we could predict the past from the future and overestimate the performance of our model
      - “We observed in the future that we had bugs in this file. Maybe we had them in the past as well.”
- How do we label data
  - The labeling also has a temporal dimension
    - The bug is only known when the issue is reported
    - The location of the bug is only known when the bug is fixed

} We look at this in greater detail

M. Tan, L. Tan, S. Dara and C. Mayeux, "Online Defect Prediction for Imbalanced Data," 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, Florence, Italy, 2015, pp. 99-108, doi: 10.1109/ICSE.2015.139.

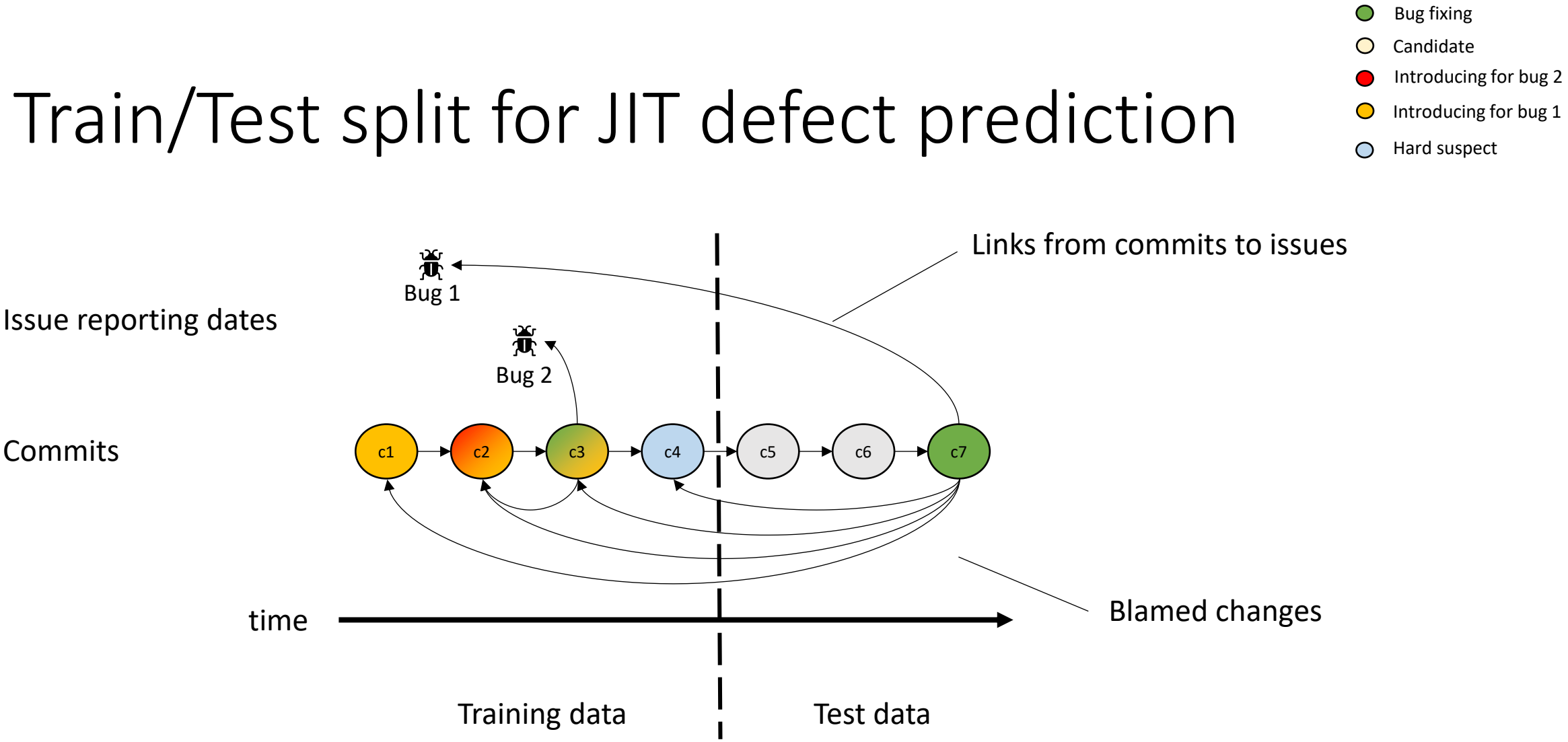
Temporal effects of SZZ



That c2 is bug introducing is only known when c3 is committed
That c1 and c3 are bug introduced is only when c7 is committed

A. Trautsch, S. Herbold and J. Grabowski, "Static source code metrics and static analysis warnings for fine-grained just-in-time defect prediction," 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), Adelaide, SA, Australia, 2020, pp. 127-138, doi: 10.1109/ICSME46990.2020.00022.

Train/Test split for JIT defect prediction



Which labels do we use for c1 and c3?!
At the time of training, we do not know that these are defective!
→ Realistic setting must use labels that are known to be noisy for training!

A. Trautsch, S. Herbold and J. Grabowski, "Static source code metrics and static analysis warnings for fine-grained just-in-time defect prediction," 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), Adelaide, SA, Australia, 2020, pp. 127-138, doi: 10.1109/ICSME46990.2020.00022.

Relationship between internal and external quality

A. Trautsch, S. Herbold and J. Grabowski, "Static source code metrics and static analysis warnings for fine-grained just-in-time defect prediction," 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), Adelaide, SA, Australia, 2020, pp. 127-138, doi: 10.1109/ICSME46990.2020.00022.

Internal vs. external quality

- Internal quality
  - Includes aspects like the readability and maintainability of the code
  - Technical debt and code smells are an aspect of internal quality
- External quality
  - Quality of the product when it is used and executed
  - Bugs are one key aspect of external quality



Smelly code

Buggy code

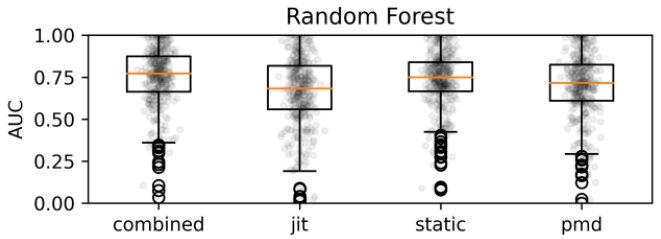
Research question: are smells (internal quality) and bugs associated (external quality) correlated?

Using JIT defect prediction to study code smells → defects relationship

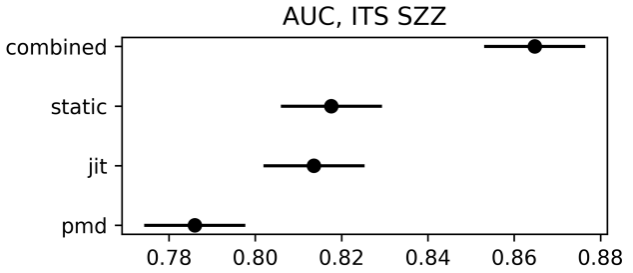
- Measure different variables about changes that are typically assumed to be relevant for the assessment if a change is defective
  - Process related metrics such as number of past changes, number of authors, etc.
  - Product related metrics such as the size and complexity
  - These are the typical metrics used as features to train JIT defect prediction model
- Measure code smells for the same code
  - Number of code smells before/after the change
  - Types of code smells that were added/removed
- Build different defect prediction models:
  - JIT: only process metrics
    - The name JIT is not very precise here, but these are the metrics commonly used in the JIT defect prediction literature and hence the namesake of this baseline
  - STATIC: only the product metrics
    - Such metrics are also known as static product metrics
  - PMD: only the code smell metrics
    - PMD is the tool used for this study
  - Combined: All of them together

Comparing JIT models

- We can then compare these models statistically
  - If the performance decreases, the code smells were used to overfit on the training data
  - If the performance is similar, the code smells did not have an impact
  - If the performance increases, the code smells helped to better identify defective code
- Results show that there is a statistically significant impact
  - There is a large effect size and this is also visible with other performance metrics
- This provides already a strong indicator that there is a relationship
  - Otherwise, such features should not help to predict defects



Boxplot with jittered points that show the actual performances within experiments



Confidence intervals of the mean values computed with Tukey's HSD test

Looking at feature importance

- Most machine learning models allow for some insights into how results were produced

- Random forest can compute the *feature importance*
  - This is essentially a measure about how often a certain feature was used to make decisions about the class
  - A high feature importance should mean that a feature had a large impact on the model performance
  - This should be controlled for correlations, which we do not discuss here

- The results show that la (lines added) is most important, but that the warning density (WD) of PMD is also very important

→ Size is the most important factor for risk, but smells matter!

Metric	Feature importance for the random forest
la (jit)	0.0143
file/system/WD (pmd)	0.0101
system/WD (pmd)	0.0101
add (jit)	0.0099
author/delta/WD (pmd)	0.0096
ld (jit)	0.0095
exp (jit)	0.0094
oexp (jit)	0.0085
entropy (jit)	0.0085
sctr (jit)	0.0078

Costs of defect prediction

S. Herbold, "On the Costs and Profit of Software Defect Prediction," in IEEE Transactions on Software Engineering, vol. 47, no. 11, pp. 2617-2631, 1 Nov. 2021, doi: 10.1109/TSE.2019.2957794.

Steffen Tunkel, Steffen Herbold: Exploring the relationship between performance metrics and cost saving potential of defect prediction models, Empirical Software Engineering, 2022

Cost Factors

Input:
Software



Output:
Prediction of defects



Introduction of a prediction model (INIT)
Maintenance of the model
Running predictions (EXEC)

Reaction:
Correction of defects



Costs for quality assurance (QA)
Costs for correction

Result:
Less post-release defects



Costs through missed defects (DEF)

Or as a formula: $cost = cost_{INIT} + cost_{EXEC} + cost_{QA} + cost_{DEF}$

Costs for correction can be ignored, as they are required to be spent regardless of whether defects are predicted or reported by users

Initial and Execution Costs

Input:
Software



Output:
Prediction of defects



Introduction of a prediction model
Maintenance of the model
Running predictions

Determined by tool market

Independent of predictions!



Modelled as constants!

$$cost_{INIT} = C_{INIT}$$

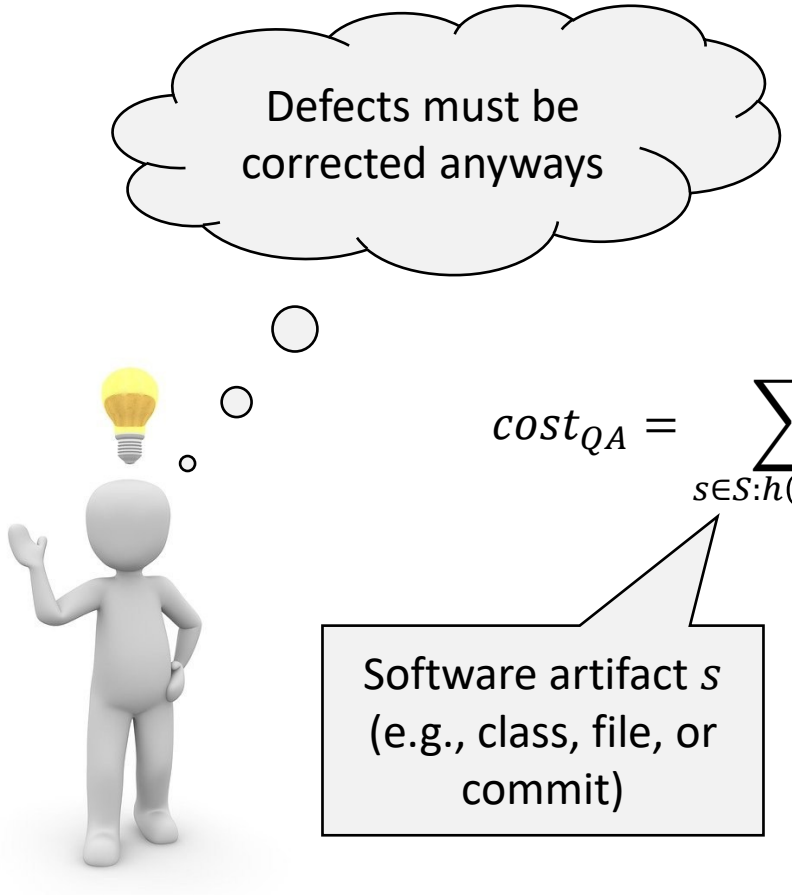
$$cost_{EXEC} = C_{EXEC}$$

Costs for Finding and Fixing Defects

Reaction:
Correction of defects



Costs of quality assurance
~~Costs of correction~~



$$cost_{QA} = \sum_{s \in S : h(s)=1} qa(s)$$

Quality assurance costs for s (e.g., code review)

Software artifact s (e.g., class, file, or commit)

Predicted as defective

Costs of Defects

Result:
Less post-release bugs



Costs through missed bugs

Missed = Not found
or not predicted



Loss due to defect d

Probability of not finding d

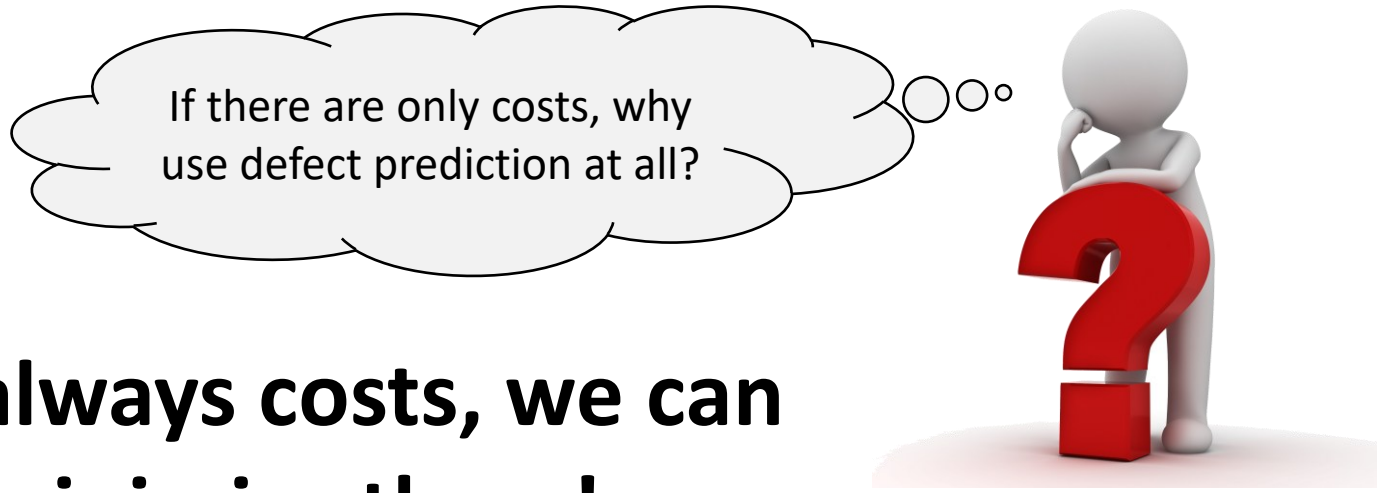
$$cost_{DEF} = \sum_{d \in D_{MISS}} loss(d) + \sum_{d \in D_{PRED}} qf(d) \cdot loss(d)$$

d not predicted

d predicted

General Cost Model

$$cost = C_{INIT} + C_{EXEC} + \sum_{s \in S : h(s)=1} qa(s) + \sum_{d \in D_{MISS}} loss(d) + \sum_{d \in D_{PRED}} qf(d) \cdot loss(d)$$



There are always costs, we can just try to minimize them!

Trivial Models

Same QA for everything

$$cost_{all} = \cancel{C_{INIT} + C_{EXEC} +} \sum_{s \in S : \cancel{h(s)=1}} qa(s) + \cancel{\sum_{d \in D_{MISS}} loss(d) +} \sum_{d \in D_{PRED}} qf(d) \cdot loss(d)$$

No quality assurance

$$cost_{none} = \cancel{C_{INIT} + C_{EXEC} + \sum_{s \in S : h(s)=1} qa(s) +} \sum_{d \in D_{MISS}} loss(d) \cancel{+ \sum_{d \in D_{PRED}} qf(d) \cdot loss(d)}$$

Calculate profit through cost comparison

Profit in relation to same QA for everything:

$$profit = cost_{all} - cost$$

$$= \sum_{s \in S : h(s) = 0} qa(s) - C_{INIT} - C_{EXEC} - \sum_{d \in D_{MISS}} \big(1 - qf(d)\big) \cdot loss(d)$$

Saved QA costs

Costs for defects that are now missed

In general, in the article:

Theorem 1. *Let S be the artifacts of a software product with post-release defects D, $h : S \to \{0, 1\}$ a defect prediction model, $qa(s)$ the costs for the quality assurance of artifact $s \in S$, $qf(d)$ the expected value that quality assurances misses a defect d, $loss(d) = C$ the costs of a defect. Furthermore, let p_{qa} be the probability that quality assurance is applied to an artifact randomly.*

Let

$$x = \sum_{d \in D_{PRED}} (qf(d) - 1) + \sum_{d \in D} p_{qa}^{|d|} \cdot (1 - qf(d)) \quad (11)$$

$$y = \sum_{s \in S} p_{qa} \cdot qa(s) - \sum_{s \in S : h(s) = 1} qa(s) - C_{INIT} - C_{EXEC} \quad (12)$$

The defect prediction model h has an expected positive profit in comparison to the random selection of artifacts with probability p_{qa}, if

$$C < \frac{y}{x} \text{ if } x > 0 \quad (13)$$

respectively

$$C > \frac{y}{x} \text{ if } x < 0 \quad (14)$$

Estimation of $qa(s)$ and $loss(d)$

Quality assurance costs per line of code

$$qa(s) = C_{QA} \cdot size(s)$$

Usually unknown ☹

Mean costs per defect

$$loss(d) = C_{DEF}$$

Can be considered as ratio!

$$C = \frac{C_{DEF}}{C_{QA}} = \frac{C_{DEF}}{1} \quad \Rightarrow \quad qa(s) = size(s)$$

$$loss(d) = C$$

Defect costs per “size unit”

And if I do not know C?



Mandatory Condition

Defect prediction is only better than a trivial model, if

$$\frac{\sum_{s \in S: h(s)=1} size(s)}{\sum_{d \in D_{PRED}} (1 - p_{qf})^{|d|}} < C < \frac{\sum_{s \in S: h(s)=0} size(s)}{\sum_{d \in D_{MISS}} (1 - p_{qf})^{|d|}}$$

Only requires a rough estimation of C to know if cost saving is possible!



Model Performance Metrics

precision

recall

G-measure

NofI-80%

F-measure

NofB-20%

Matthews Correlation Coefficient

Area Under the LOC/Bugs Curve

Area Under the Curve



No ☹

RQ: Are metrics and costs correlated?!

Model Performance Metrics

precision

recall

G-measure

NofI-80%

F-measure

NofB-20%

Matthews Correlation Coefficient

Area Under the LOC/Bugs Curve

Area Under the Curve



No ☹

Relationship?!

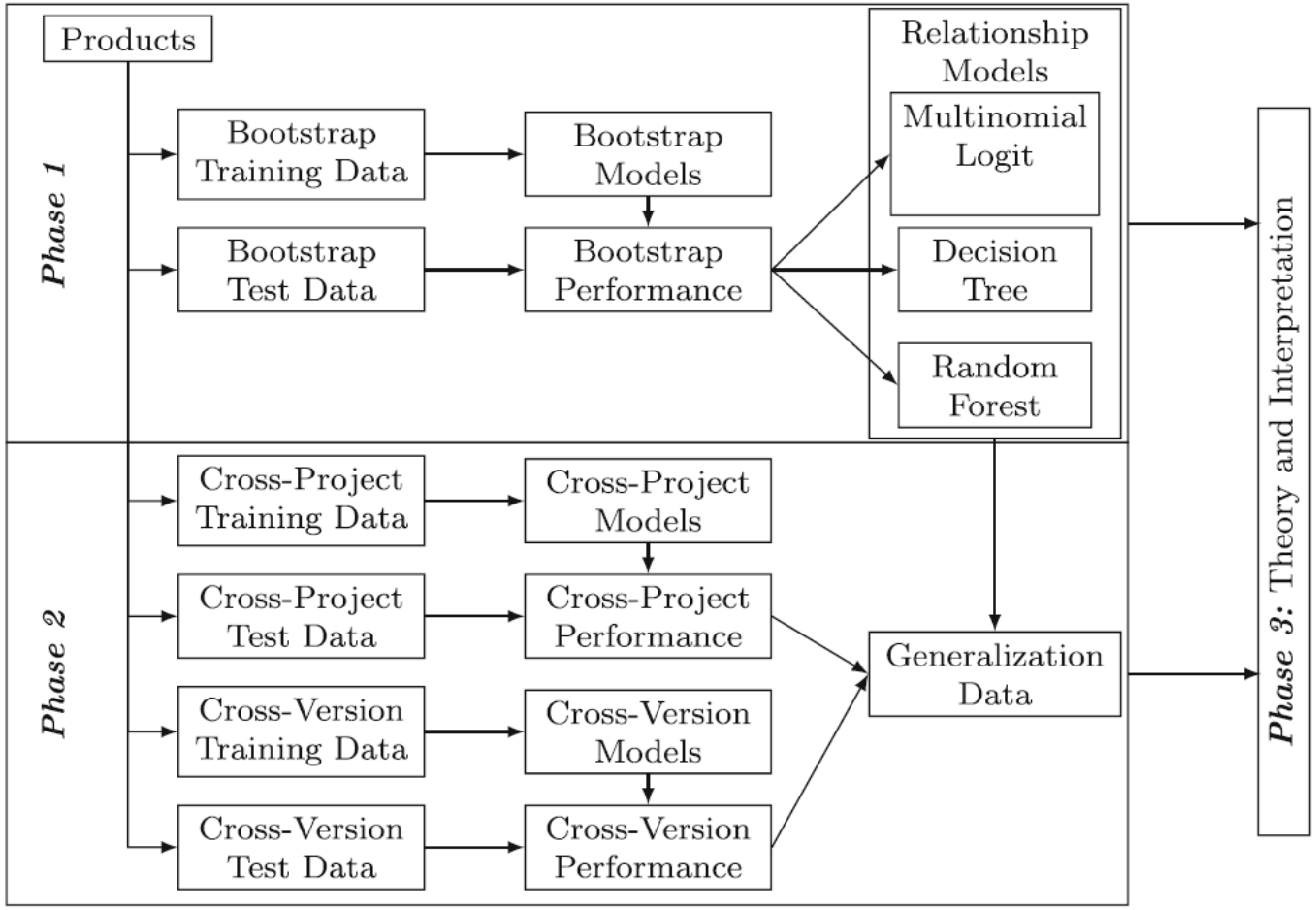
Mandatory Condition

Defect prediction is only better than a trivial model, if

$$\frac{\sum_{s \in S: h(s)=1} size(s)}{\sum_{d \in D_{PRED}} (1 - p_{qf})^{|d|}} < C < \frac{\sum_{s \in S: h(s)=0} size(s)}{\sum_{d \in D_{MISS}} (1 - p_{qf})^{|d|}}$$

C are the defect costs per "size unit"

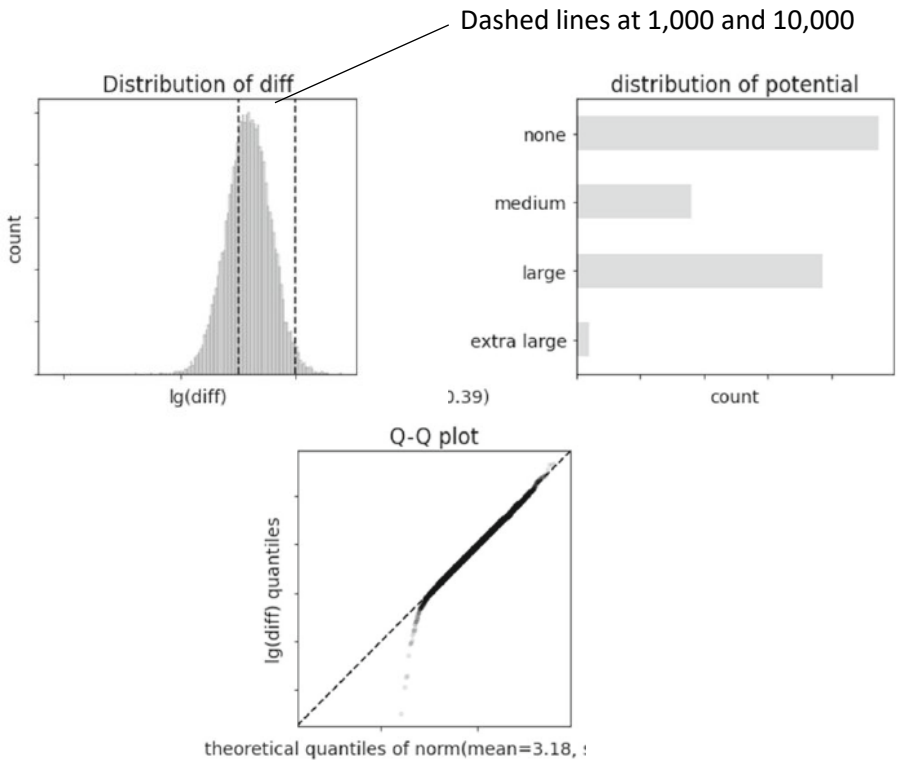
Approach



Builds two kinds of models:

- Within-project models based on bootstrap samples
  - Used to model relationship between metrics and costs
- Cross-project/version models
  - Cross-project: use data from other projects
  - Cross-version: use old releases from same project
  - Used to determine if modelled relationship generalizes

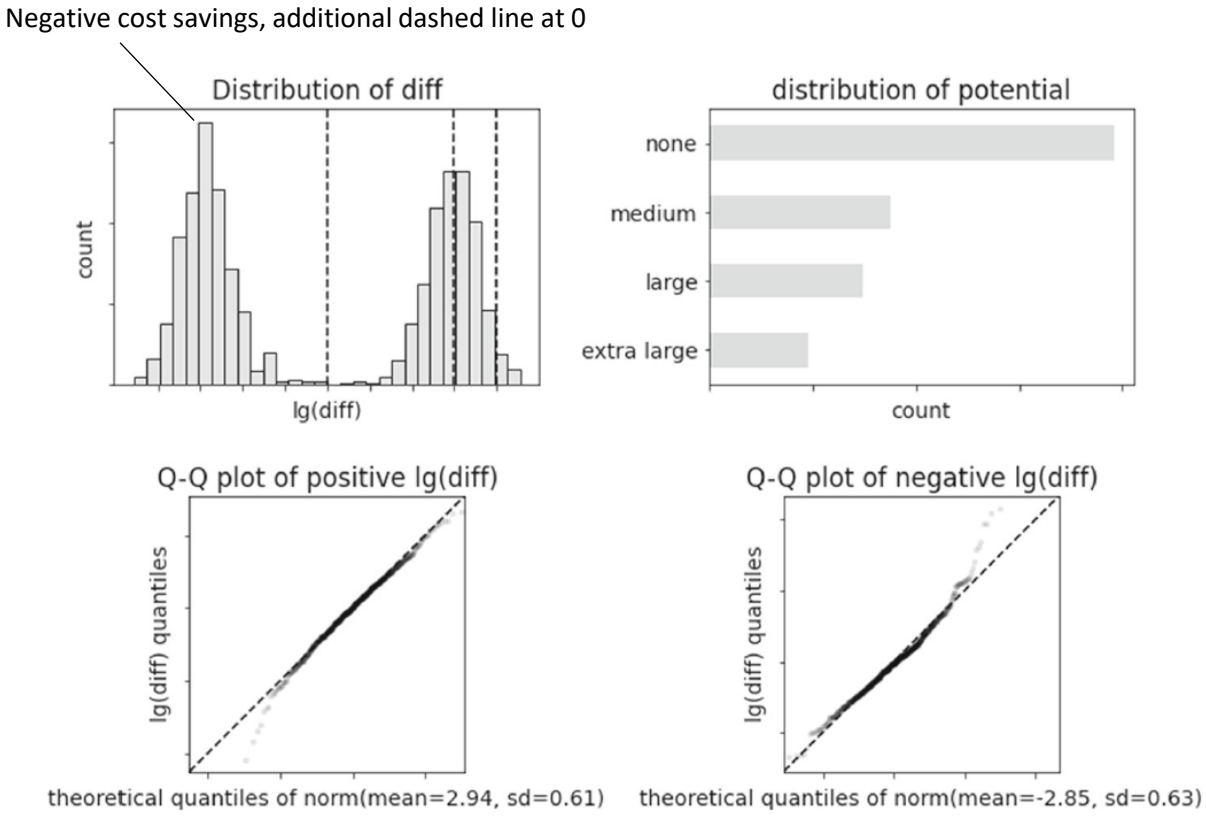
Bootstrap data



Log-normal distributions of cost saving potential

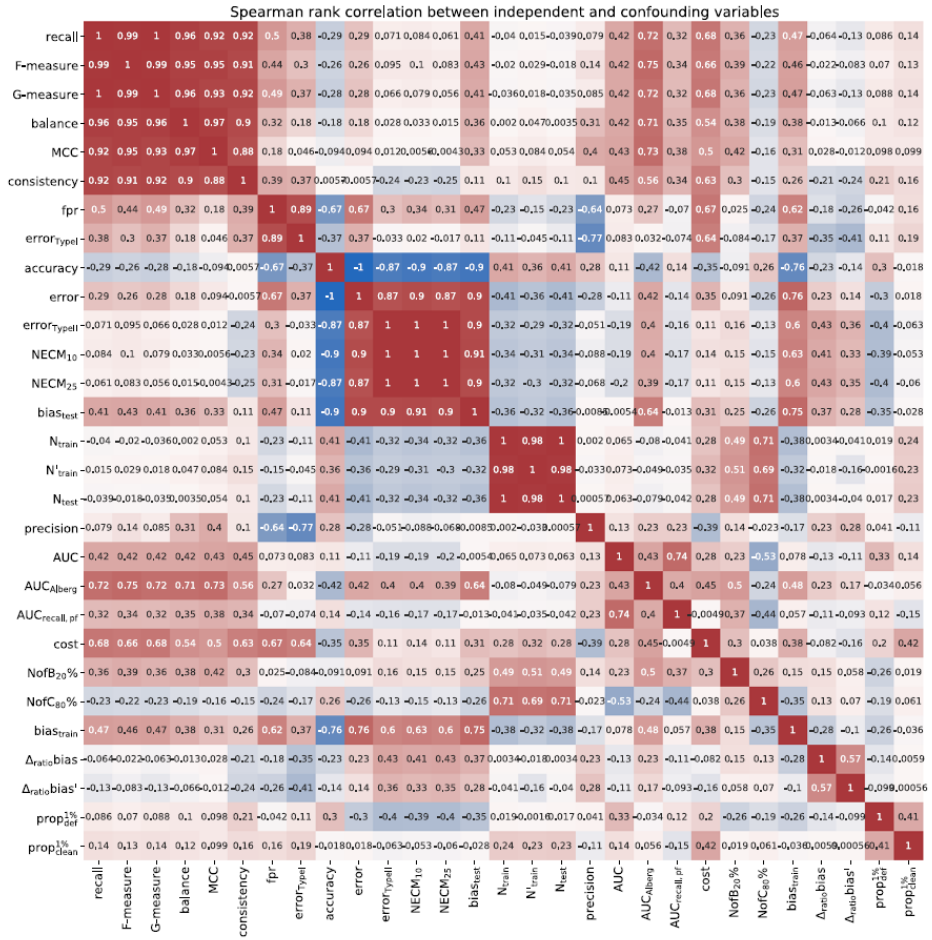
- Mean at about 1000 LOC
- Almost no negative cost savings
  - Models without cost-saving potential (none) are typically trivial models, e.g., predicting no defects at all

Realistic settings

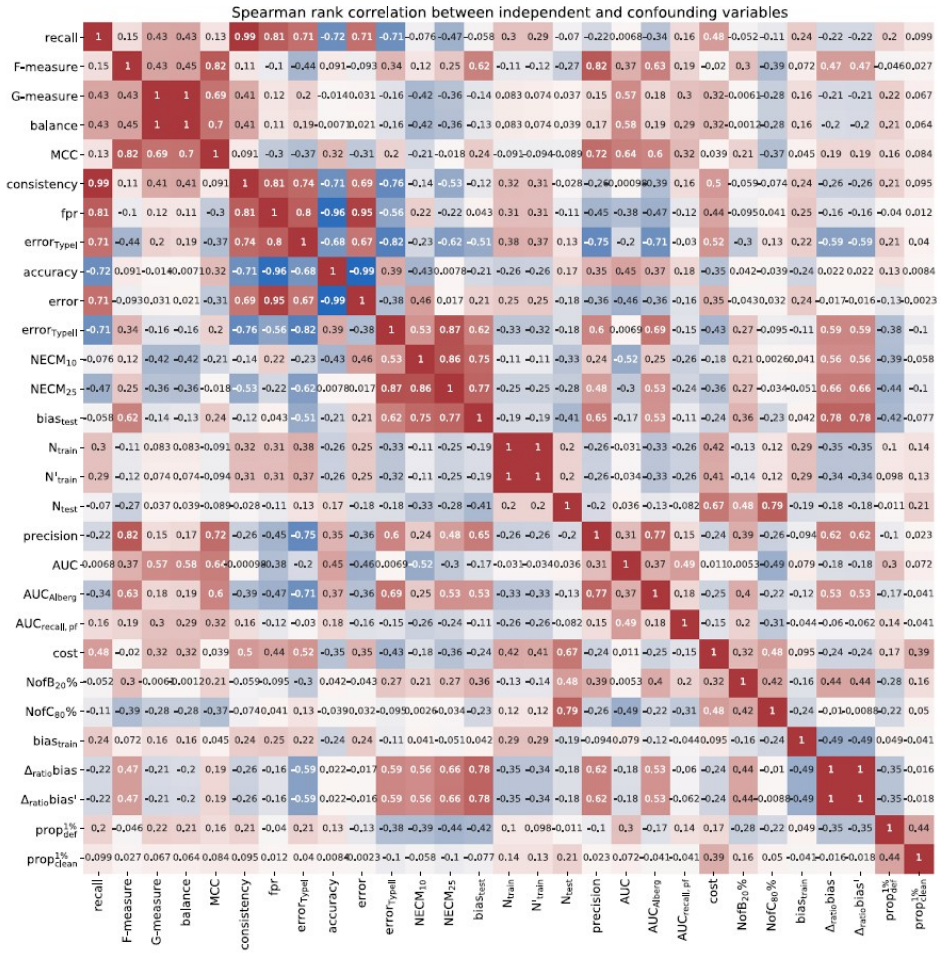


In general similar, but now we have a mirrored log-normal distribution for negative cost-saving potential, i.e., losses

Bootstrap data

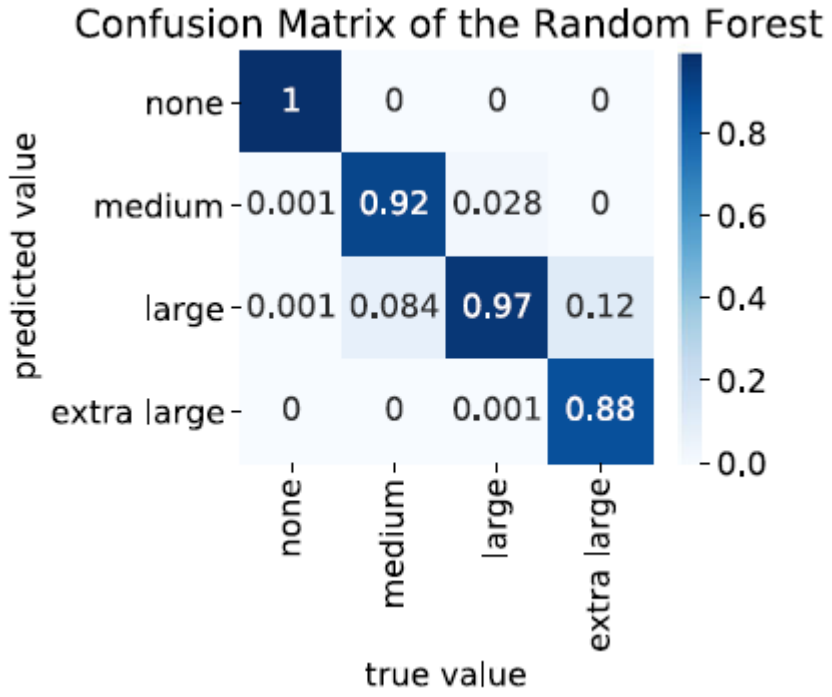


Realistic settings



Correlations between variables do not generalize!

Relationship in bootstrap data

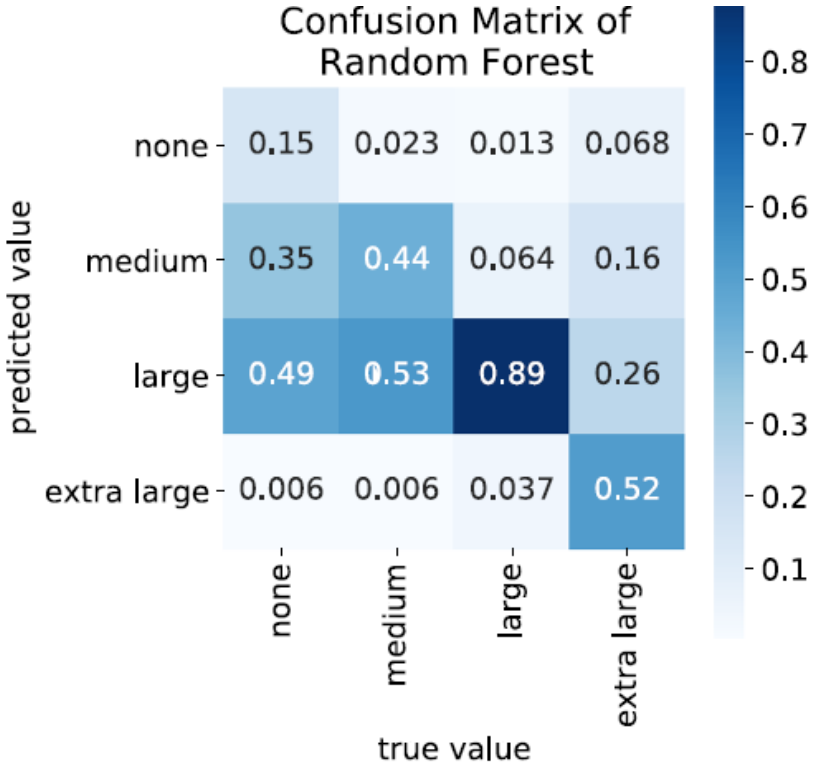


Works!

But knowledge strongly correlated with confounder: bias in test data

- Here, bias means class level imbalance, i.e., there are much fewer defective artifacts than clean ones

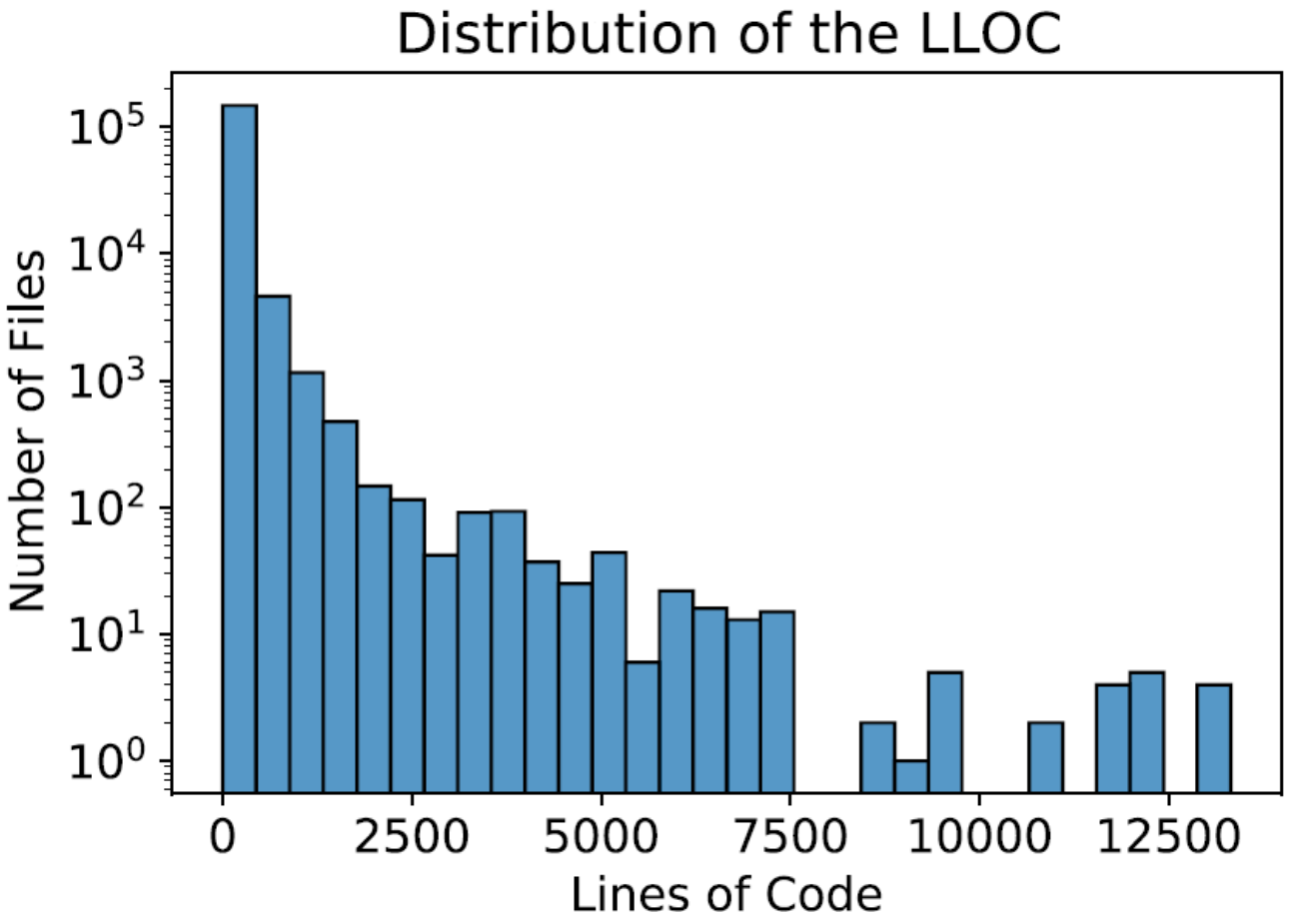
Generalization of relationship



Does not work!

Also does not work between cross-version and cross-project!

Reason for lack of generalization



Because costs vary strongly between instances!

Only cost modeling captures this!

The other metrics (mostly) rely on the confusion matrix which assumes that

- all missed bugs costs the same
- all quality assurance measures cost the same

The cost model computes the costs for both individually, based on size

- Size is a proxy here for the costs, i.e., an indirect measure which comes with some threats to validity

Questions?

